OpenMP — Lesson 3

Fixing the race: `critical`, `atomic` and `reduction`

Study notes for Flor Vargas

November 2026

Sources: Chapman, Jost & van der Pas [1], Ch. 4 "OpenMP Language Features" (the `critical` and `atomic` constructs, the `reduction` clause) and Ch. 5 "How to Get Good Performance" (cost of synchronization).

1 The idea

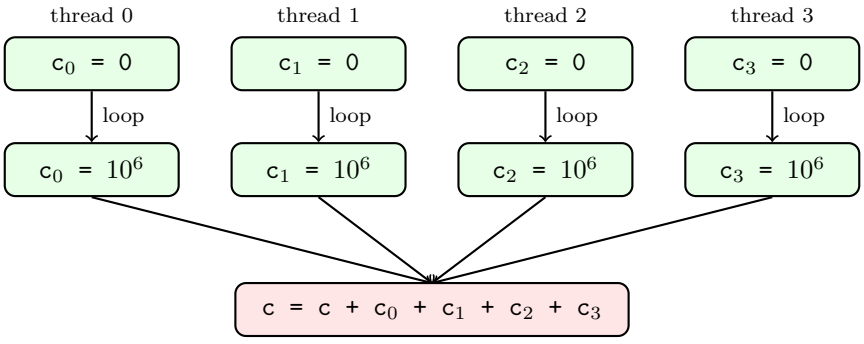
In Lesson 2 several threads incremented a shared `counter` and increments were lost. There are two ways to fix it:

1. **Coordinate the threads** so that only one updates the variable at a time: `critical` or `atomic`. Correct, but threads must wait for each other.
2. **Stop sharing**: each thread accumulates into its own private copy, and the copies are combined once at the end: `reduction`. Correct and with no waiting.

2 The three tools

Tool	What it guarantees	Use it for
`#pragma omp critical`	one thread at a time runs the block (a lock)	any code: several statements, calls
`#pragma omp atomic`	the next single update is indivisible	one statement: `x++`, `x += e`
`reduction(op:x)`	private copy per thread, combined with `op` at the end	sums, products, max, min

How `reduction(+:c)` works with 4 threads:



done once, safely, at the end of the region

The private copy is initialised to the *identity* of the operator: 0 for `+`, 1 for `*`.

3 The code

The same counter as Lesson 2, written four times: the race, then each fix. Each version is timed with `omp_get_wtime()`, which returns wall-clock seconds.

fix_race.c

```c
/*
 * fix_race.c -- OpenMP Lesson 3: three ways to update a shared variable safely.
 * Source: Chapman, Jost & van der Pas (2007), "Using OpenMP",
 *         Ch. 4 (critical and atomic constructs, reduction clause),
 *         Ch. 5 (performance cost of synchronization).
 *
 * Compile (no -O flag, as in Lesson 2):
 *     gcc -Wall -fopenmp -o fix_race fix_race.c
 * Run:
 *     OMP_NUM_THREADS=4 ./fix_race
 */

#include <stdio.h>
#include <omp.h>

#define N 1000000    /* increments done by EACH thread (same as Lesson 2) */

int main(void)
{
    int nthreads = omp_get_max_threads();   /* team size the regions will use */
    double t;

    printf("threads = %d, expected = %d\n\n", nthreads, nthreads * N);

    /* ---- 0) The bug from Lesson 2: unprotected shared update ---- */
    int c0 = 0;
    t = omp_get_wtime();                    /* wall-clock time in seconds */
    #pragma omp parallel
    {
        for (int i = 0; i < N; i++)
            c0++;                           /* DATA RACE */
    }
    printf("race      : %9d   %.4f s\n", c0, omp_get_wtime() - t);

    /* ---- 1) critical: only ONE thread at a time may run the block ----
       Works for any code (several statements, function calls, I/O).
       Cost: threads wait in line for a lock on every increment.        */
    int c1 = 0;
    t = omp_get_wtime();
    #pragma omp parallel
    {
        for (int i = 0; i < N; i++) {
            #pragma omp critical
            c1++;
        }
    }
    printf("critical  : %9d   %.4f s\n", c1, omp_get_wtime() - t);

    /* ---- 2) atomic: the single update x++ / x += e is done indivisibly ----
       Only for ONE simple update statement, but uses a hardware
       instruction instead of a lock, so it is cheaper than critical.   */
    int c2 = 0;
    t = omp_get_wtime();
    #pragma omp parallel
    {
        for (int i = 0; i < N; i++) {
            #pragma omp atomic
            c2++;
```

```
        }
    }
    printf("atomic    : %9d   %.4f s\n", c2, omp_get_wtime() - t);

    /* ---- 3) reduction: each thread gets a PRIVATE copy of c3, set to 0
       (the identity of +). At the end of the region OpenMP adds all the
       private copies into the shared c3. No synchronization in the loop. */
    int c3 = 0;
    t = omp_get_wtime();
    #pragma omp parallel reduction(+:c3)
    {
        for (int i = 0; i < N; i++)
            c3++;                           /* private copy: no race */
    }
    printf("reduction : %9d   %.4f s\n", c3, omp_get_wtime() - t);

    return 0;
}
```

```
gcc -Wall -fopenmp -o fix_race fix_race.c
OMP_NUM_THREADS=4 ./fix_race
```

Typical output (your times will differ):

```
threads = 4, expected = 4000000

race      :   2402115   0.0087 s
critical  :   4000000   0.4488 s
atomic    :   4000000   0.1637 s
reduction :   4000000   0.0019 s
```

Key ideas

- All three fixes give the correct result. They differ only in cost.
- `critical` is the most general and the most expensive: every increment takes and releases a lock, and the other threads wait.
- `atomic` is cheaper because the hardware does the update in one indivisible instruction, but it still forces threads to take turns on one memory location.
- `reduction` has no synchronization inside the loop. It is the right tool whenever the pattern is "combine many values with one operator".
- Rule of thumb: prefer `reduction` > `atomic` > `critical`.

4 The experiment

Experiment

Question: how does the cost of each fix change as the number of threads grows?

1. Run with 1, 2 and 4 threads:

```
for t in 1 2 4; do echo "== $t threads"; OMP_NUM_THREADS=$t ./fix_race; done
```

2. Write the times in a table: rows = method, columns = number of threads.

What to observe and answer.

1. With 1 thread, `critical` and `atomic` are already slower than `reduction`, although there is nobody to wait for. Why?
2. As threads are added, the work per thread stays the same but `critical` and `atomic` get *slower*. Why does adding threads make them worse?
3. Why does `reduction` stay fast?

Answers (read after trying). (1) Synchronization has a fixed cost even with no competition: taking and releasing a lock, or executing a locked hardware instruction, is much slower than a plain `c++`. We pay it 10^6 times. (2) More threads means more of them competing for the same lock or memory location (*contention*). The updates are forced into a single line, so the program runs essentially serially, plus the overhead of handing the variable from one core to another. (3) Each thread works only on its private copy, so the threads never wait for each other. The single combining step at the end costs almost nothing. Lesson: the fastest synchronization is the one you avoid.

Next lesson. Sharing the *work*: so far every thread did all N iterations. The `for` work-sharing construct splits one loop's iterations among the threads (Chapman et al., Ch. 4).

References

[1] B. Chapman, G. Jost, and R. van der Pas. *Using OpenMP: Portable Shared Memory Parallel Programming*. MIT Press, Cambridge, MA, 2007.